可编程调度之openEuler实践：构建精准算力调度新底座

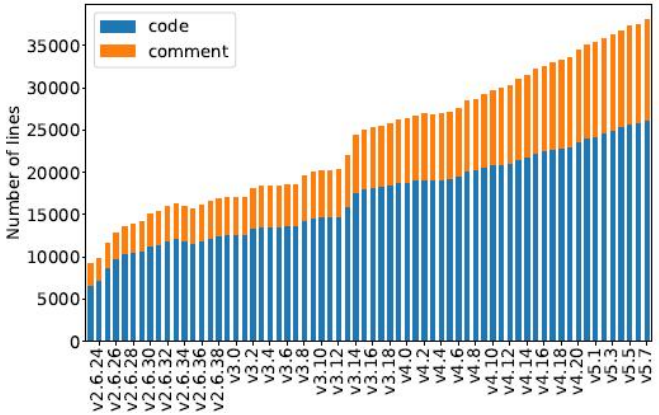
屈子程、杨宸

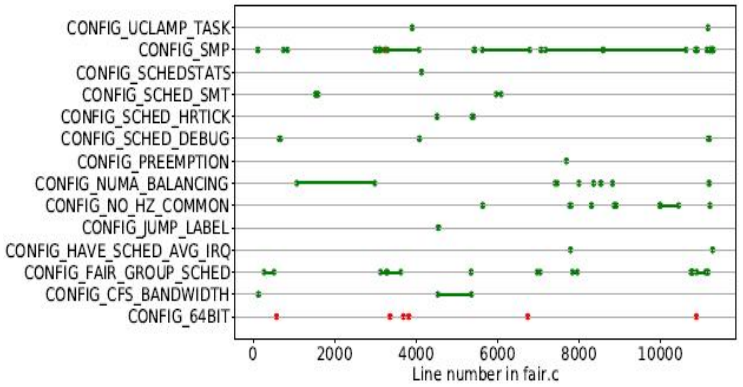
- 可编程调度发展现状
- openEuler可编程调度实践
- openEuler可编程调度未来工作

可编程调度发展现状

可编程调度发展背景

应用性能需求的多样性：延迟 vs 吞吐





通用调度器的维护挑战：调度设计演进，复杂度高；2003 O(1) -> 2007 CFS -> EEVDF 2023

2004年

Con Kolivas提交Pluggable cpu scheduler framework patch

2005年

法国inria Julia Lawall教授团队提出基于事件模型的用户态和内核态协同调度开发框架Bossa

2020年

EuroSys20:法国inria Julia Lawall基于Bossa提出内核机制分离的调度协同增强调度开发框架Ipanema(多核与负载机制支持等)

2021年

SOSP21:Google提出用户态策略和内核机制协调度开发框架ghOst

2024年

Eurosys24: 华盛顿大学提出基于rust的调度开发框架Enoki

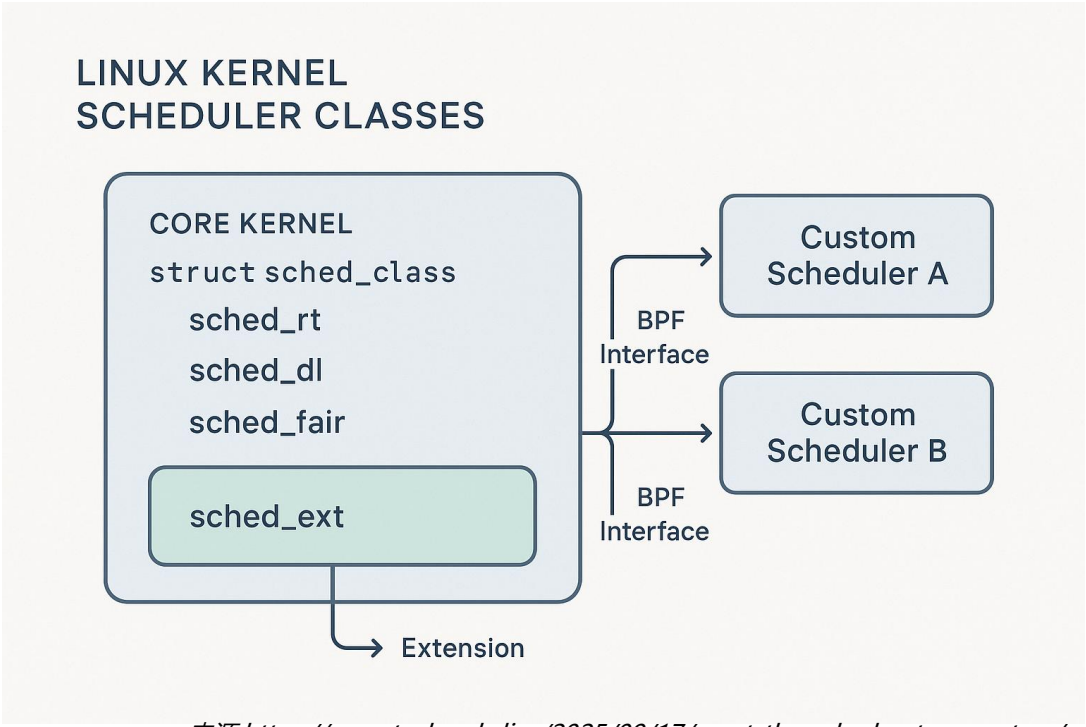
Linux v6.12:社区合入基于eBPF的内核调度开发框架sched_ext

2026年

- 能效与拓扑感知全面增强
- 性能与稳定性生产级优化
- 可观测性与调试体系完善

开放原子开源基金会 OPENATOM FOUNDATION | OpenEuler

可编程调度框架—Linux sched_ext



来源:https://www.techveda.live/2025/09/17/meet-the-sched_ext-ecosystem/

- **原理介绍**
  - **策略与机制分离：**内核仅负责底层的核心调度机制（如上下文切换、状态管理、锁处理），而将“谁该运行”和“在哪运行”的决策权委派给 BPF 程序。
  - **分发队列架构 (DSQ)：**引入了 Dispatch Queues (DSQs) 作为中转站。BPF 调度器通过 select_cpu 和 enqueue 等钩子，将任务放入不同的 DSQ（如局部、全局或自定义队列）。
  - **安全性与性能平衡：**利用 BPF 验证器确保自定义调度代码不会导致系统崩溃。系统会通过 Watchdog 自动卸载它并回退到默认的 CFS/EEVDF 调度器，确保系统始终稳定。
- **主要特征**
  - **跨态语义融合与应用感知调度：**打通用户态与内核态语义鸿沟，让底层调度策略精准感知并契合上层业务模型，榨取系统最佳性能。
  - **高效能的策略生命周期管理：** 提供高效的编程框架，显著降低了调度策略的开发门槛，打通了“开发-验证-部署”全流程，有效缩短技术迭代周期。
  - **高可用容错与原生内核兜底机制：**提供内核级别的无缝熔断保障，在自定义策略发生异常时，系统可自动回退至默认内核调度器，确保全局业务的高可靠运行。

开放原子开源基金会 OPENATOM FOUNDATION | OpenEuler

Linux 可编程调度框架sched_ext业界探索

Meta：大规模生产环境验证

- **scx_lavd (通用/延迟敏感):** 默认调度方案，兼顾通用性vs时延定制化
- **scx_layered (AI 训练加速): 消除**算子下发干扰，**算力利用率提升 4%**

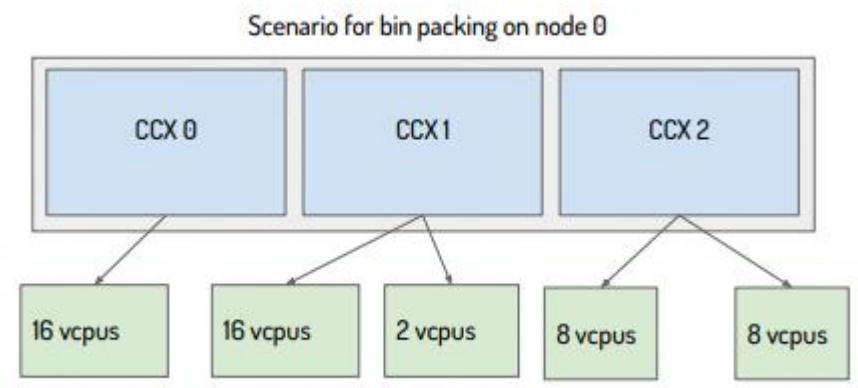
scx_layered → Bucket the tasks in the system into layers with different properties and scheduling policies:

Latency critical tasks + comms monitoring and signaling	Reduce preemptions, run on cores with higher frequency, preserve cache locality, etc.
Data pipeline tasks	Allow variable CPU count usage, can run on cores at lower frequency, limit memory bandwidth usage, etc.
Supporting frameworks tasks	Confine into a fixed number of CPUs but sufficient enough to avoid stalls
The rest	Whatever is left

https://lpc.events/event/19/contributions/2039/attachments/1872/4121/%5BLPC%5D%20Accelerating_AI_Training_With_Sched_Ext%20v3.pdf

Google：云环境与架构优化

- **scx_user (AMD Zen 架构优化):** 实现 CCX Cache 感知的用户态调度，specJBB +108%，RT_Bench +35%。



https://lpc.events/event/19/contributions/2034/attachments/1883/4108/lpc_chiplet_aware_sched_policy.pdf

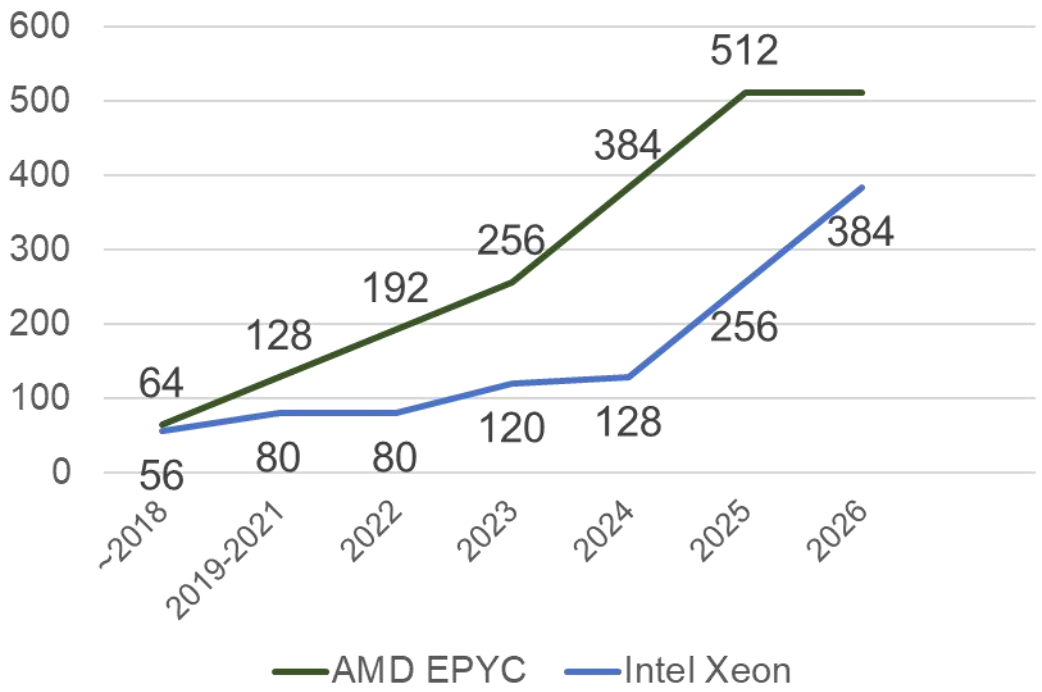
更多开源用户态调度器，详见sched_ext开源仓库: https://github.com/sched-ext/scx/tree/main/scheds/rust

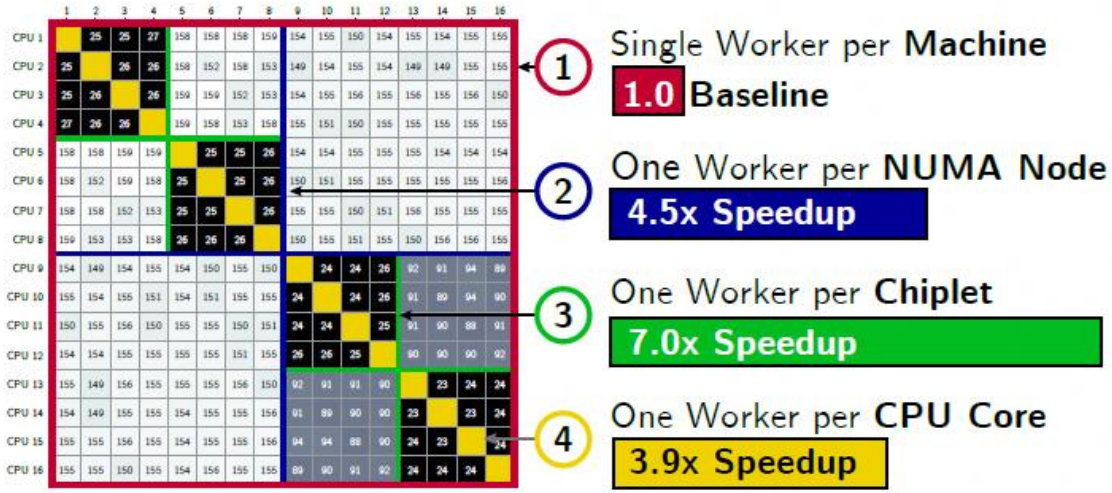
开放原子开源基金会 OPENATOM FOUNDATION | OpenEuler

openEuler可编程调度实践

众核背景介绍

- **众核架构：** 芯片向众核、Multi-chiplet（如 AMD的CCD/CCX，鲲鹏多cluster）演进
- **多 LLC 时代：** 跨 Die / 跨 LLC 的数据访问延迟惩罚呈指数级上升，Cache Miss 和数据同步带来的时延可能成为新的瓶颈





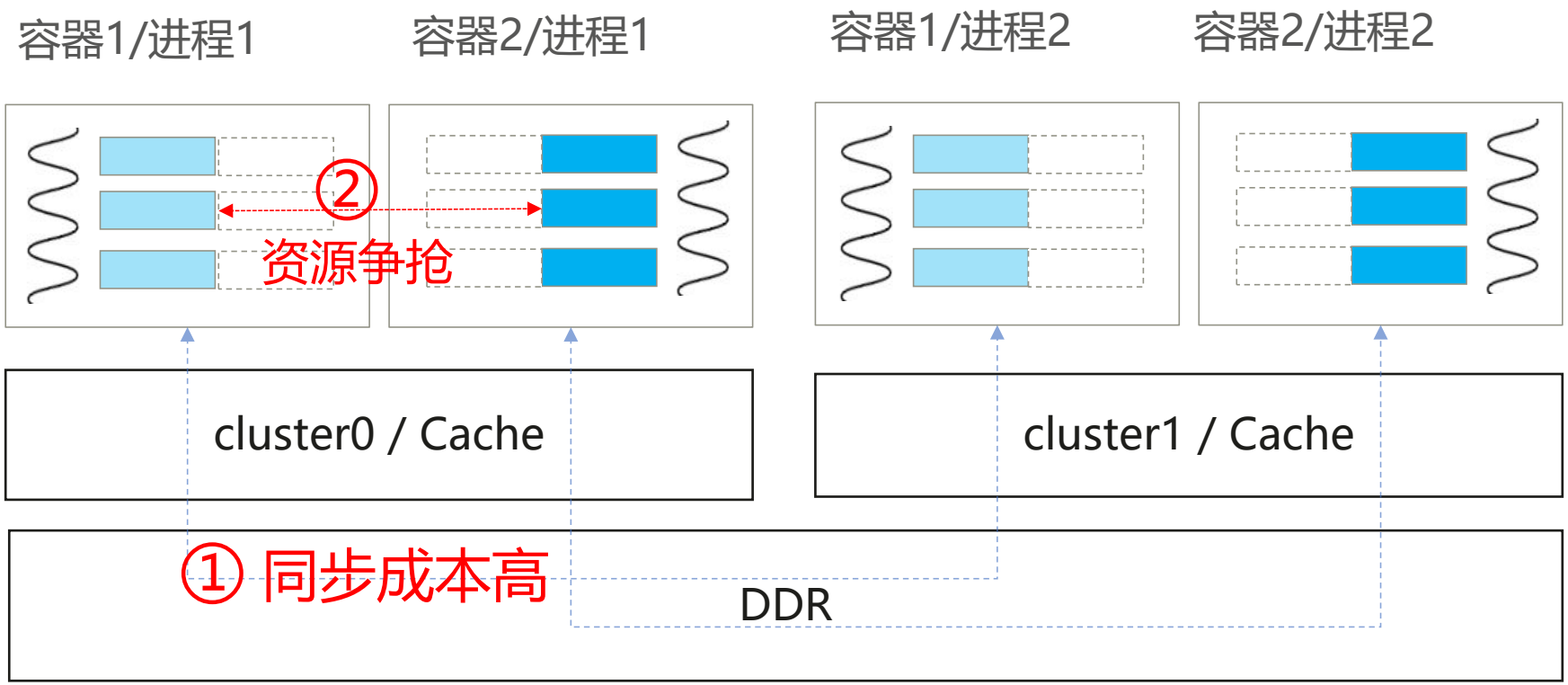
(a) Core-to-core latency (AMD Ryzen 1950) (b) Speedup (TPC-H)

Fogli, A., Zhao, B., Pietzuch, P., Bandle, M., & Giceva, J. (2024). OLAP on modern chiplet-based processors. *Proceedings of the VLDB Endowment*, *17*(11), 3428-3441.

传统调度的众核设计挑战

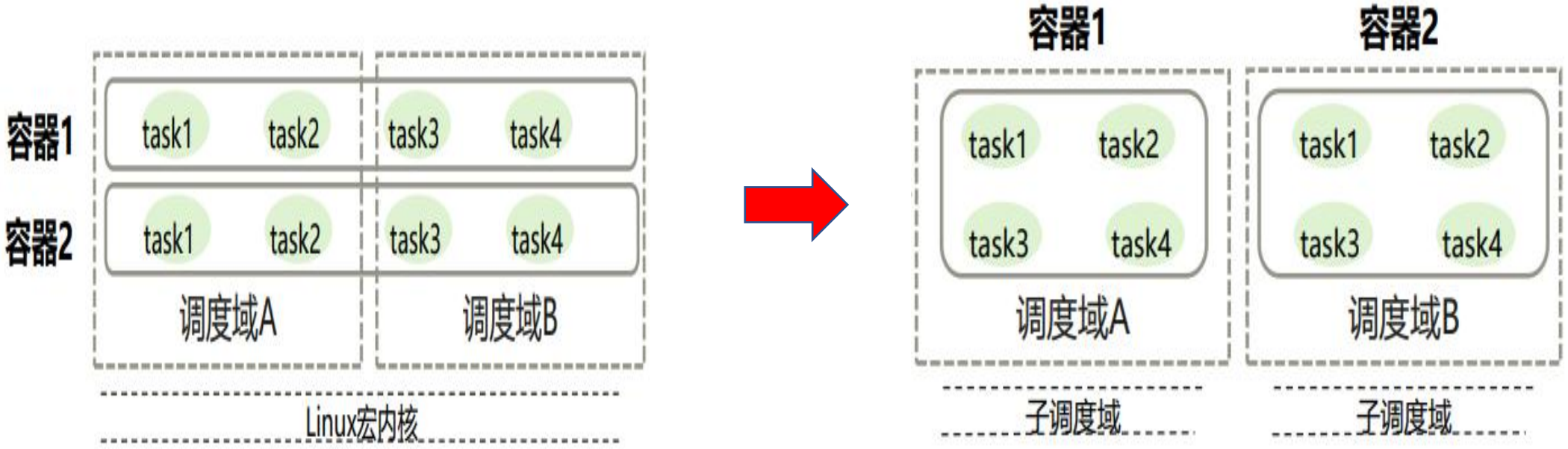
LinuxCFS/EEVDF：缺乏“业务语义”，导致 Cache 颠簸

- **微观视角的盲区**： 内核调度以线程为粒度，不感知进程 / 容器边界
- **负载均衡的副作用：** 频繁的 load_balance 将高频交互的线程打散至不同 LCC
- **性能灾难：** 数据跨 Cache 通信激增，引发业务的吞吐量暴跌、时延抖动剧烈



面向chiplet架构的众核域级调度

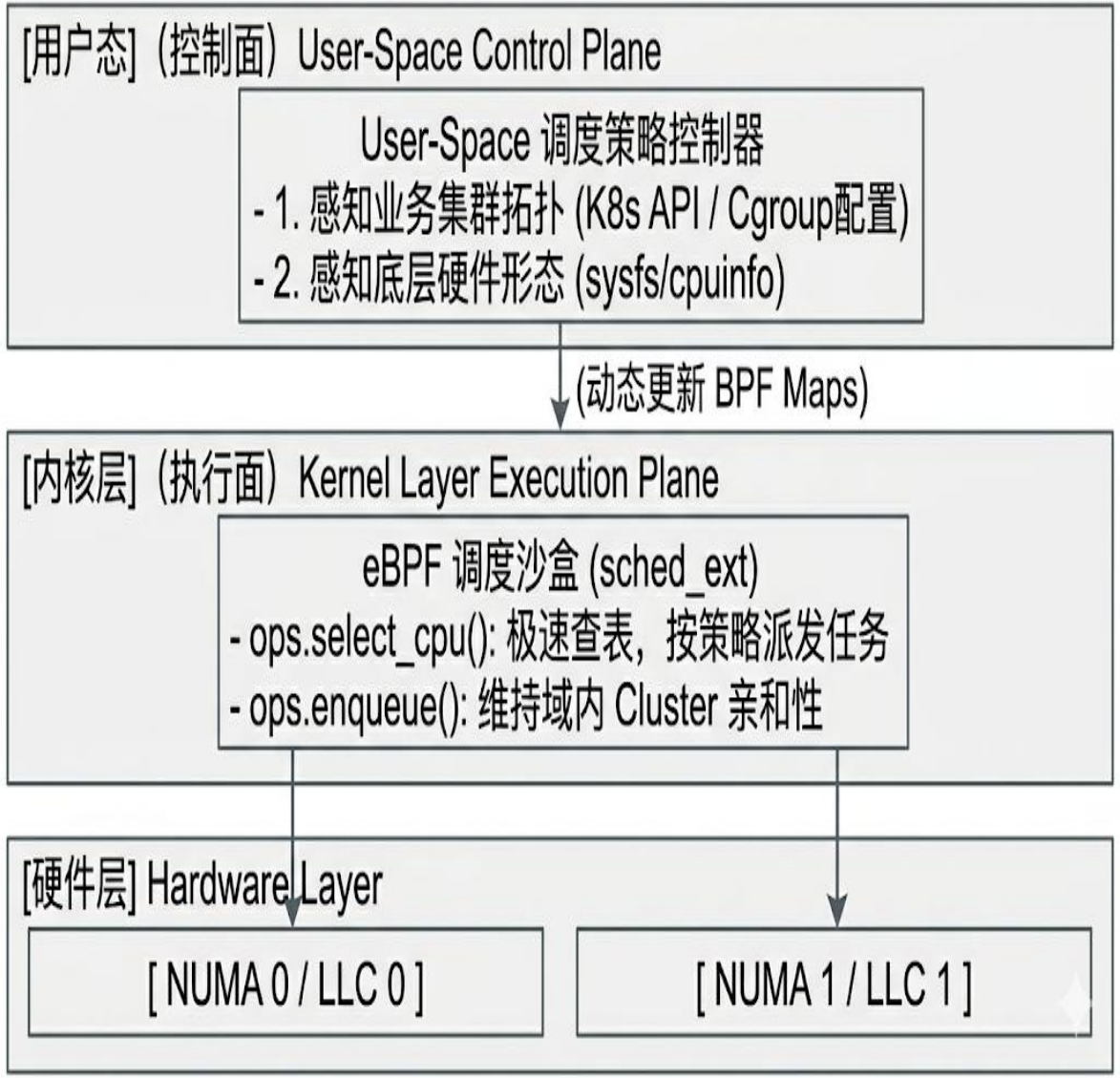
分域调度：以容器/任务组视角，基于cgroup配置感知容器级业务关系，关联任务优先聚合调度到对应的优先调度域（如 LLC/Cluster）



均衡调度：容器任务在多个调度域分布

分域调度：以调度域为”一等公民”编排容器任务

分域调度可编程实践



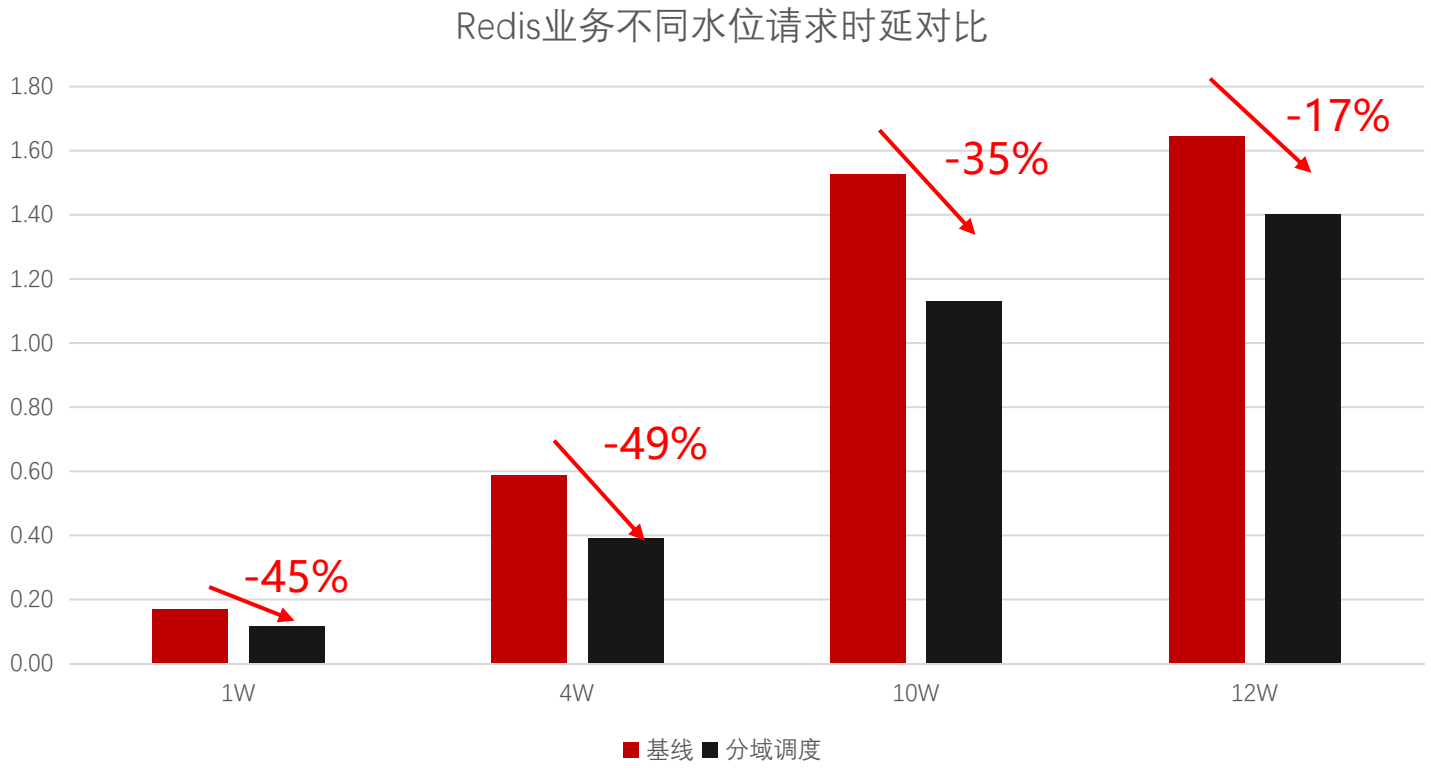
User-space调度策略控制器

- 基于cgroup配置感知业务关系
- 通过sysfs接口构建CPU拓扑关系
- 动态生成 [任务组 ↔ 优选调度域] 映射矩阵

sched_ext调度策略

- **ops.select_cpu():**基于 BPF Map 中预计算的「任务组→调度域」映射矩阵极速查表，O(1) 复杂度返回最优 调度域
- **ops.enqueue():** 任务放入特定域内的本地队列，维持同一容器/进程线程的域内亲和性

• 效果验证



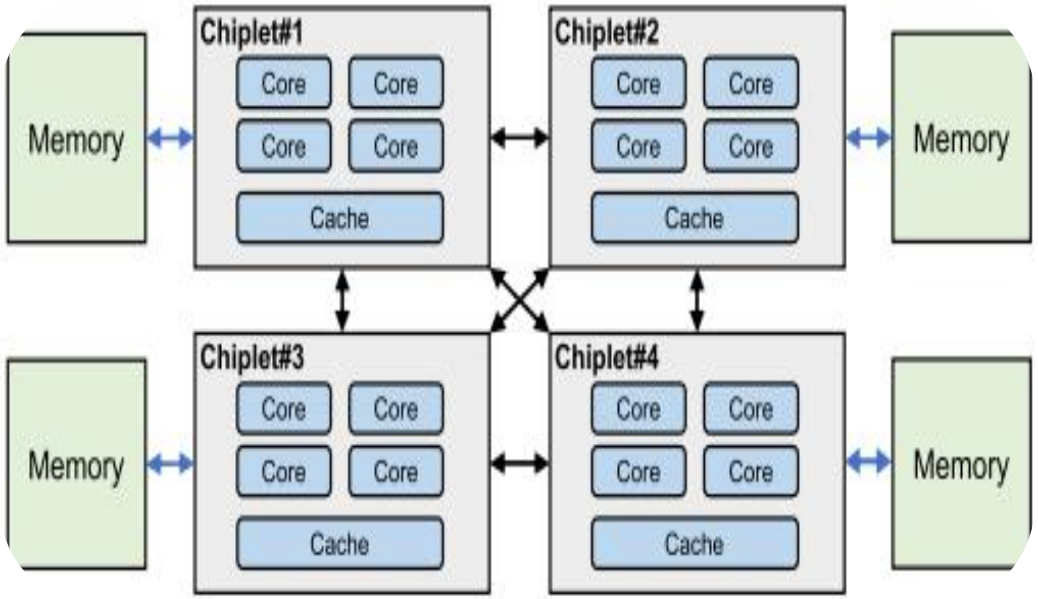
redis业务场景:

分域调度优化平均时延降低17%~45%，p99时延降低13%~87%

开放原子开源基金会 OPENATOM FOUNDATION | OpenEuler

NUMA调度背景

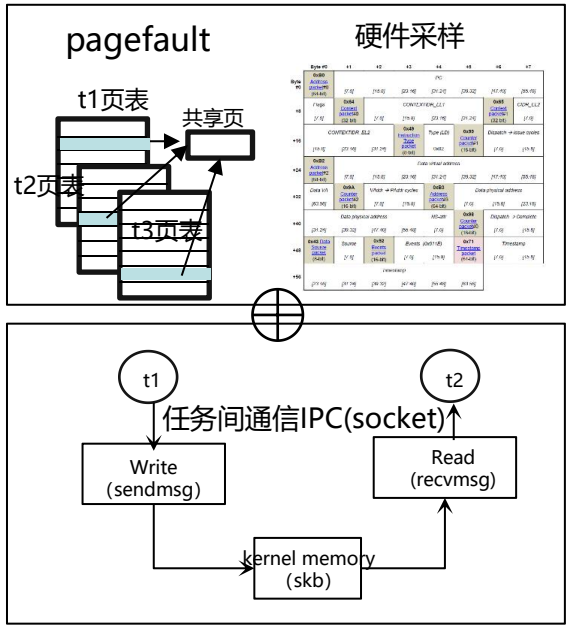
- **NUMA（非统一内存访问）成为主流内存架构，如何做好数据排布、算力调度，成为提升系统性能关键**
  - **内核调度面向SMP设计**：强调核间负载均衡和数据Affinity，任务Affinity不足
  - **现有机制决策收敛周期长**：内核AutoNUMA，基于pagefault统计页访问和节点位置信息，通过静态阀值控制fault扫描rate，采样底噪不可控且反馈时间长(扫描时间和VM的大小：256MB/s)



开放原子开源基金会 OPENATOM FOUNDATION | OpenEuler

面向NUMA亲和调度

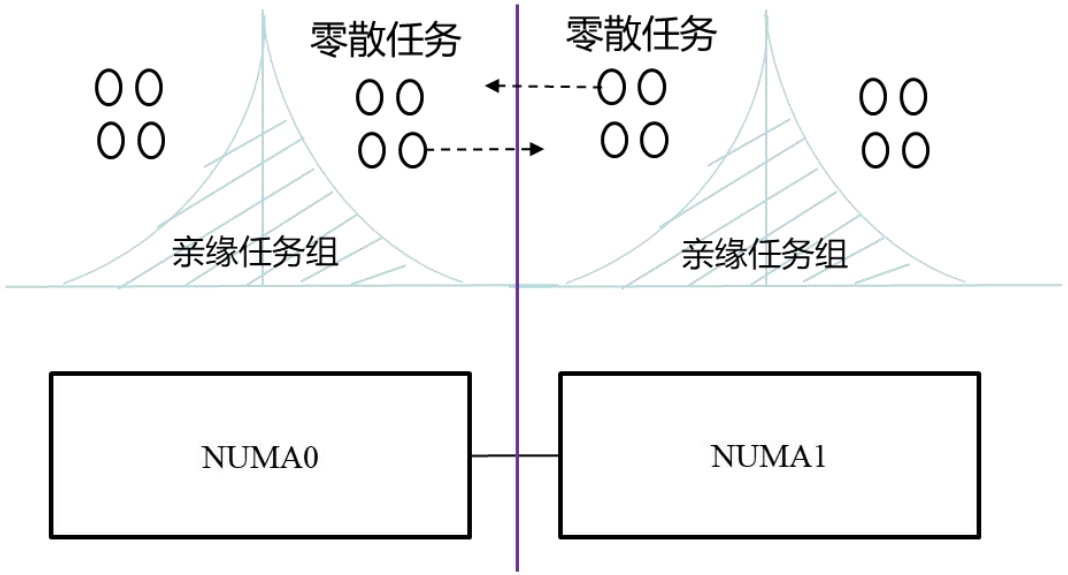
- **自适应NUMA**: 根据任务亲缘关系，调整NUMA选核与负载均衡机制，实现任务动态聚合



Affinity矩阵

Affinity 权重	t1	t2	t3
t1	-	10	30
t2	10	-	40
t3	30	40	-

强弱	t1&t2	t2&t3
亲缘	弱	强



开放原子开源基金会 OPENATOM FOUNDATION | OpenEuler

自适应NUMA可编程实践

scx_numa调度器：

- 创建ebpf 亲缘关系组和CPU MAP表；
- 基于访存大小、频率和网络交互次数实现周期性衰减策略;
- 通过kfunc获取负载、带宽、干扰信息，实现资源瓶颈策略，决定NUMA内或NUMA间调度；

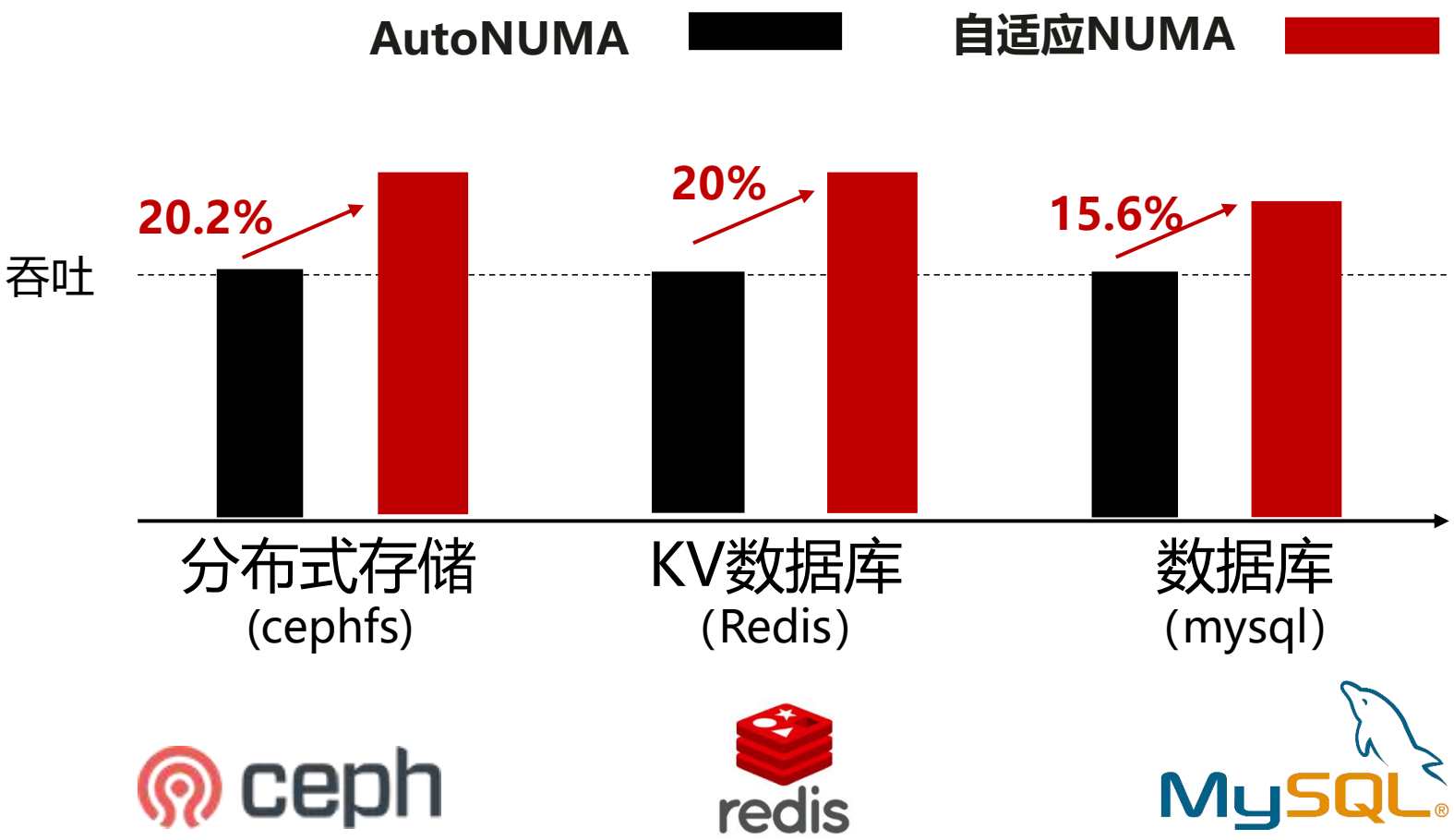
sched_ext内核机制：

ops.select_cpu：基于亲缘关系MAP，按亲缘瓶颈策略和亲缘关系强弱，调整关系组CPU MAP表；

ops.enqueue：从CPU MAP表获取CPU范围，优先选择本地队列。

开放原子开源基金会 OPENATOM FOUNDATION | OpenEuler

效果验证



开放原子开源基金会 OPENATOM FOUNDATION | OpenEuler

openEuler可编程调度未来工作

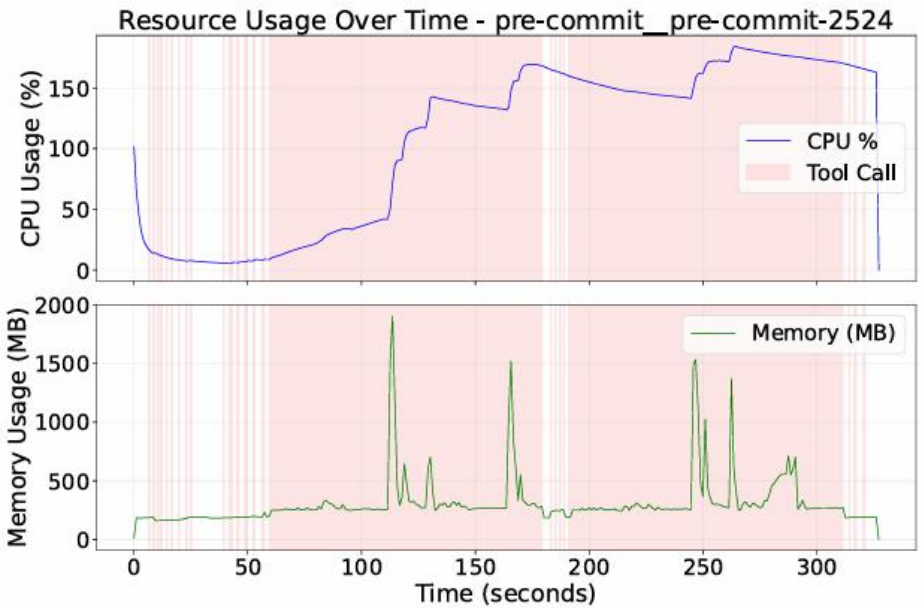
面向Agent场景下意图驱动的资源精细化调度探索

问题与背景

Agent负载特征下资源管理的三大错位：粒度太粗/响应过慢/适应性

	Static Limits	Reactive Control	Predictive Scaling
Tools	mem.max/high, cpu.max [14]; K8s QoS [19]	PSI [35]; oomd [26, 31]; TMO [36]	VPA [20]; Autopilot [29]
Assumes	Known peak; stable demand	Gradual pressure; kill acceptable	Repeatable; history valid
Agent	15.4× peak/avg; tool-semantic variation	1–2 s burst; unpredictable timing	1.8× variance; kill = lose context
Mismatch	Granularity	Responsiveness	Adaptability

Agent下突发内存使用对调度/内存带宽控制提出更高诉求

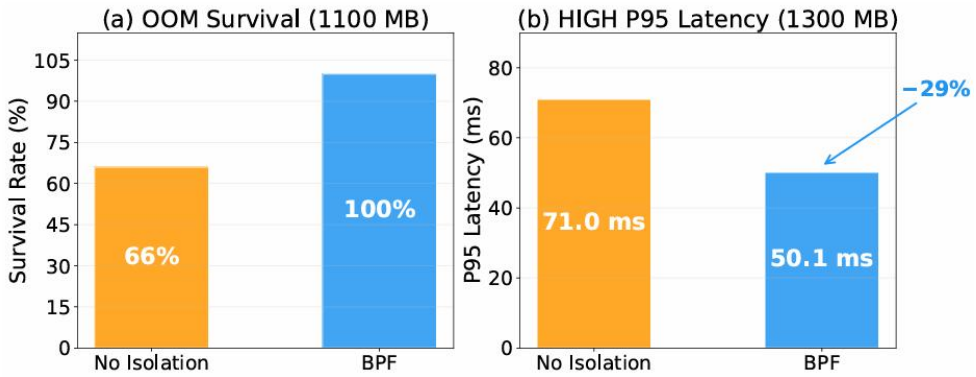


技术方案

基于sched_ext&memcg_bpf，实现意图驱动的精细化资源管理

- **sched_ext**：维护Tool调用负载特征，优先处理时延敏感任务；
- **memcg_bpf_ops**：监控Tool的内存负载，实现自适应限速&分级管理；
- **意图驱动的资源协商**：超限场景注入自然语言反馈，指导Agent下一步行为；

效果



https://arxiv.org/abs/2602.09345

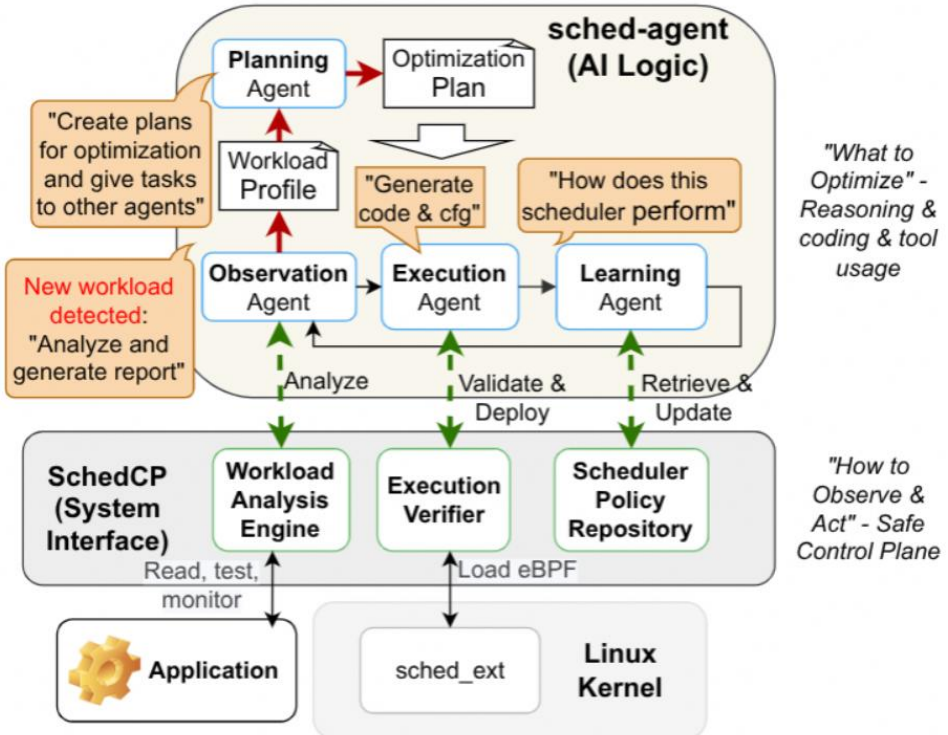
调度策略自适应生成与演进，提升Agent场景下资源供给精准度与灵敏度

问题与背景

- **可编程调度专业性要求高。**可编程调度能力虽然降低了编程和部署的难度，但是调度以及ebpf的专业性，仍然使得可编程调度无法普及；
- **手动切调度器繁琐复杂。**系统负载动态变化，手动调整切换调度器非常繁琐，响应迟滞，难以满足瞬时响应与自适应的要求；

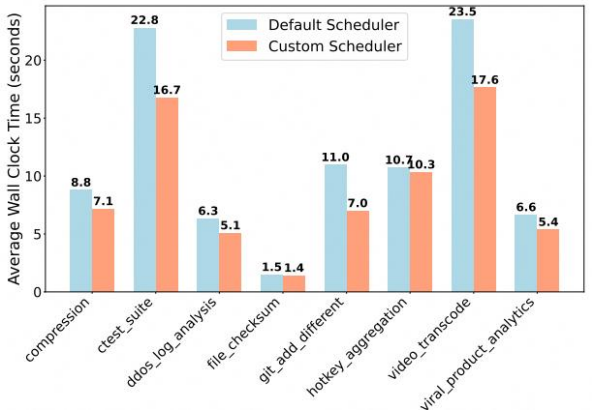
技术方案

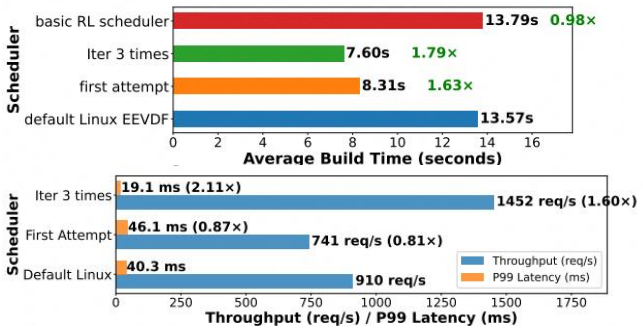
- **工作负载分析引擎(Workload Analysis Engine)**： 自动识别运行的任务特性（是时延敏感型、吞吐量导向型还是交互型）。
- **调度策略库(Scheduler Policy Repository)**：包含多种预置的调度器（如scx_layered等），并能根据需求动态合成新的策略。
- **执行校验器(Execution Verifier)**：AI 生成的代码或配置部署前，通过静态和动态分析进行验证，防止 AI 弄崩内核。



感知 -> 决策 -> 执行 -> 学习

效果

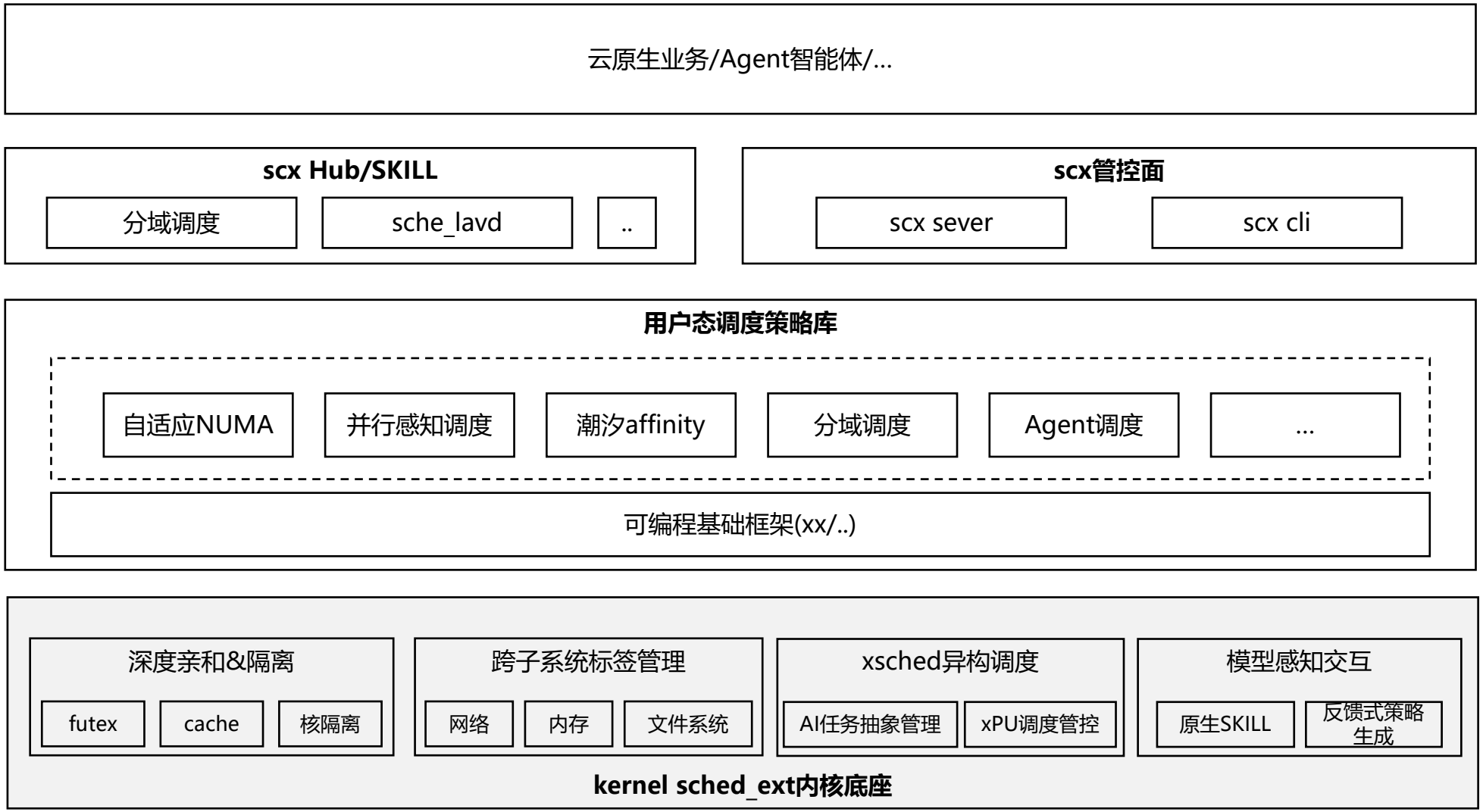




https://github.com/eunomia-bpf/schedcp

开放原子开源基金会 OPENATOM FOUNDATION | OpenEuler

聚焦完善sched_ext底座能力，依托社区构筑面向Agent可编程调度生态



业务使能&反馈

认证发布&一键式发放

丰富的场景化调度策略

内核底座能力完善

开放原子开源基金会 OPENATOM FOUNDATION | OpenEuler

THANKS

开放原子开源基金会
OPENATOM FOUNDATION | OpenEuler